

Process Concept

Module 3

Operating Systems

Chapter Outline

- Process Concept
- Program vs Process
- Process Control Block
- Process Table
- Context Switch
- Process states
- Process states transitions
- Process life cycle
- Five state process model
- Unix process state transition diagram

Process Concept

- A process is basically a program in execution
 - or that which a processor executes
- A process is unit of work in a system
- A process is an activity performed by a system
 - It may be program's data, program counter, pointer or register used in a program
- A process uses resources like CPU, memory, files and I/O devices
- Operating system processes execute system code and user processes executes user code

Program vs Process

- A program is a passive entity
- A process is a dynamic entity represented by the values stored in registers, memory, program counters etc.
- A program does not compete for the computing resources whereas a process does
- One program can have several processes
 - Consider multiple users executing the same program

Process Concept

- Multiple parts
 - The program code, also called **text section**
 - Current activity including **program counter**, processor registers
 - **Stack** containing temporary data
 - Function parameters, return addresses, local variables
 - **Data section** containing global variables
 - **Heap** containing memory dynamically allocated during run time

Process Concept

- Process idea was first used in Multics
- All programming OS, such as Windows NT, or MVS (multiple virtual storage) were built around the concept of process
- Major requirements that the OS must meet can all be expressed with reference to process
 - OS must interleave the execution of a number of processes to maximize processor use
 - OS must allocate resources to process in conforming to a specify policy
 - e.g. certain functions or applications are of higher priority
 - at the same time avoiding deadlock
 - OS may support inter-process communication

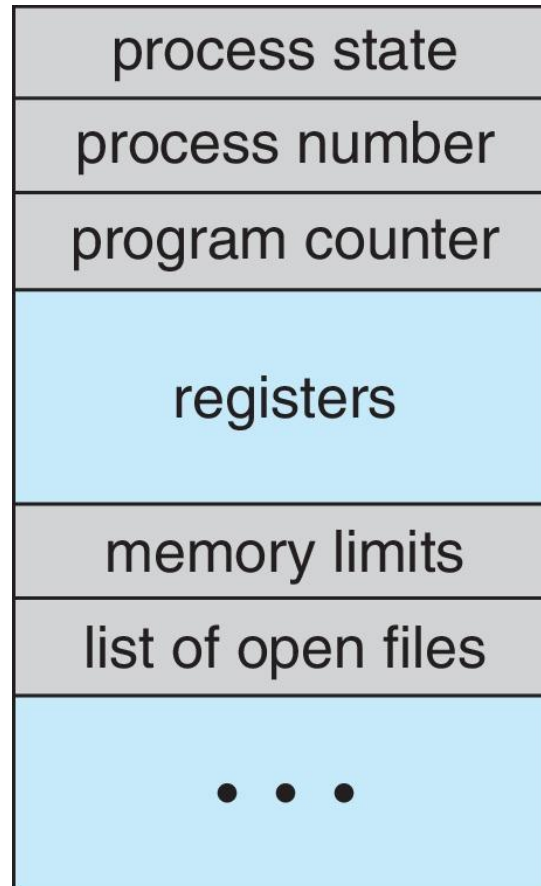
Process Control Block (PCB)

- When a new user program is initiated
 - OS creates a data structure, called Process Control Block (PCB), which gives substance to the process and serves to control it
- Each process is represented by its own process control block
- PCB is a data block or record containing information associated with a specified process
- Allows the OS to locate all key information about a process
- When the OS switches the attention of the CPU among various active processes
 - It uses the same area in PCB to hold the information
 - That information used to restart process when the process next gets CPU
- Process control block is also called task control block

Process Control Block (PCB)

- PCB contains following information about process
 - Current state of the process
 - Unique identification of process
 - A pointer to process parent
 - Pointer to process's child processes
 - The process's priority
 - Pointer to locate the process memory
 - Pointer to allocate resources
 - The register save area – CPU register
 - The processor it is running on
 - Accounting information
 - I/O state information

Process Control Block (PCB)



Process Table

- OS maintains the process table with one entry per process
- This entry contains information about process's
 - state
 - its program counter
 - stack pointer
 - memory allocation
 - the status of its open file
 - its accounting and scheduling information
 - its priority

The Shell

- It is a command interpreter
- It is a process which starts by typing a command at prompt
- It provides an interface to the OS for the user
- A facility for online users rather than batch
- Command can be entered
 - directly at an online terminal
 - through shell program or script
 - a text file having series of commands forms a script
- DOS shell is alternative as screen oriented interface

The Shell

- In Unix when any user login in, a shell is started up
 - It starts up as soon as user types some thing at command prompt, e.g. date
 - The shell creates a child process and run the date program
 - When child is running shell waits for its termination
 - When child finishes, the shell types \$ prompt again and wait for next command

Operations on Process

- A system managing processes may perform following operations
 - Create a process
 - Destroy a process
 - Suspend a process
 - Resume a process
 - Block a process
 - Wakeup a process
 - Dispatch a process
 - Change process's priority
 - Enable a process to communicate with other processes
- For long term suspension, process's resources should be free
- Changing the priority of process normally involved modifying the priority value in PCB

Process Creation & Termination

- A process may create other processes
- When a process creates a new process
 - The creator process is called the parent process
 - Created process is called the child process
- Act of creating a process is called spawning a process
- A child process may share the resources of its parent process

Process Creation & Termination

- A parent may wait for its child process to terminate before resuming or may run concurrently with the child process
- A child process may run the same program as that of its parent or may run a separate program
- In DOS, parent and child process do not run in parallel and parent process will suspend itself till child has finished

Process Creation & Termination

- When a process is created the following operations are performed
 - Process naming (process id)
 - Creates the process control block
 - Insert it in process table
 - Determine the process initial priority
 - Allocate the process's initial resources
 - Allocate the address space to be used by the process
- When a process is destroyed following operations are performed
 - Its resources are returned to the system
 - Its process control block is erased
 - It is purged from any system list or table

Reasons for Process Creation

1. New batch job
 - In a batch environment, a process is created in response to the submission of a job
2. Interactive log on
 - A user at a terminal logs on to the system
3. Created by OS to provide a service
 - OS can create a process to perform a function on behalf of a user program, without user having to wait, e.g. printing
4. Created by existing process
 - A user program can create a number of processes
 - Typically related processes need to communicate and cooperate each other
 - Achieving this cooperation is a difficult task for the programmer of OS

Reasons for Process Termination

- There must be some means for a process to indicate its completion
 - A batch job should include a halt instruction, which generates an interrupt to alert the OS that a process has completed
 - For an interactive application, the action of the user will indicate whether the process is completed

Reasons for Process Termination

Following are the possible reasons for process termination;

1. Normal termination

- The process executes an OS service call to indicate that it has completed running

2. Time limit exceeds

- The process has run longer than the specified total time limit which includes
 - total time elapsed
 - amount of time spent executing
 - in case of interactive process, the amount of time since the user last provided any input

3. Memory unavailable

- The process requires more memory than the system can provide

Reasons for Process Termination

4. Bounds violation

- The process tries to access the memory locations that is not allowed to access

5. Protection error

- The process attempt to use a file that is not allowed to use or,
- It tries to use it in improper fashion, such as writing to a read only file

6. Arithmetic error

- The process tries a prohibited computation such as division by zero

7. Time overrun

- The process has waited longer than specified maximum for a certain events to occur

Reasons for Process Termination

8. I/O failure

- An error occur in input or output, such as
 - inability to find a file
 - failure to read a write after a specified maximum no of tries (when, e.g. a defective area is encountered on a diskette) or
 - invalid operation (such as reading from the line printer)

9. Invalid instruction

- The process attempts to execute a non-existent instruction (often a result of branching into a data area and attempting to execute data)

10. Privileged instruction

- The process attempts to use an instruction reserved for the operating system

Reasons for Process Termination

11. Data misuse

- A piece of data is of the wrong type or is not initialized

12. OS or operator intervention

- For some reason, the operator or the OS has terminated the process (e.g. if a deadlock exists)

13. Parent termination

- When a parent terminates, the OS should be design to automatically terminate all the offspring of that parent

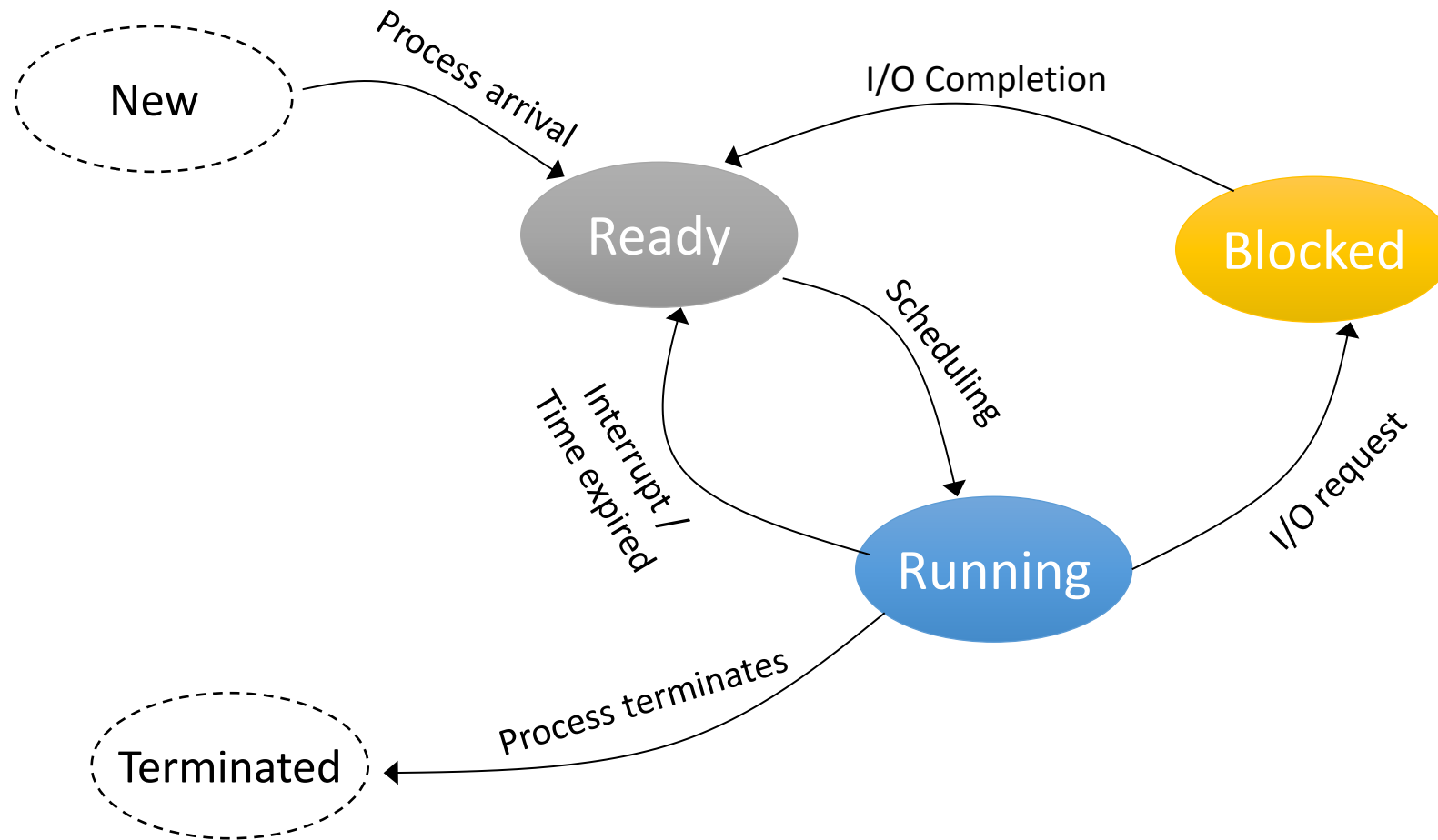
14. Parent request

- A parent process typically has the authority to terminate any of its offspring

Process States

- A process may have the following three states
 - Running
 - Instructions are being executed
 - Waiting/Blocked
 - The process is waiting for some event to occur (may be I/O completion)
 - Ready
 - The process is waiting to be assigned to processor
- At any instant a process is in one and only one of three states
- Process entering the system must go initially into the ready state
- Process can only enter the running state via the ready state
- Many OS are constructed using only these three states
- However, there is good justification for adding additional states to the model

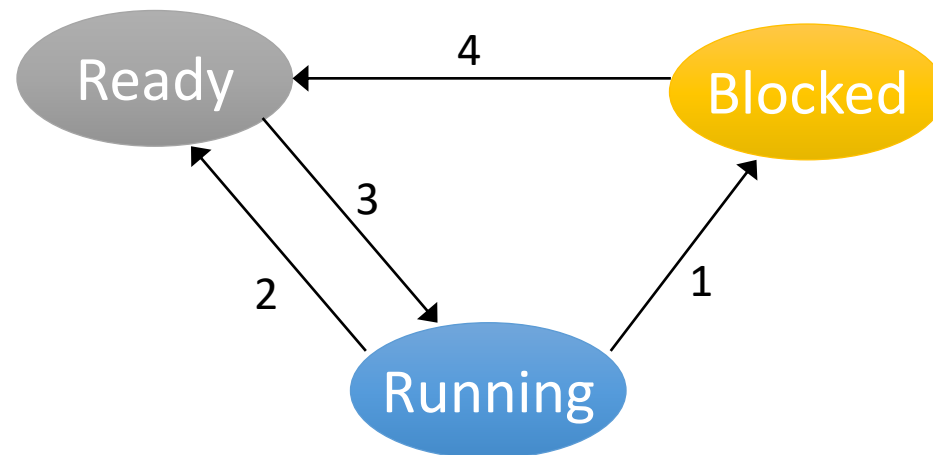
Process States



Process State Transitions

- Transition-1:
 - Occurs when a process cannot continue may be due to I/O request
- Transition-2:
 - Request when scheduler decides that the running process has run long enough , and it's the time to give CPU to other processes in ready state
- Transition-3:
 - Occurs when all the other process have had their CPU time and its time for the first process to run again
- Transition-4:
 - Occurs when external event for which process was waiting happens. If no other process is running at that time, transition-3 will be triggered and the process will start execution

Process State Transitions

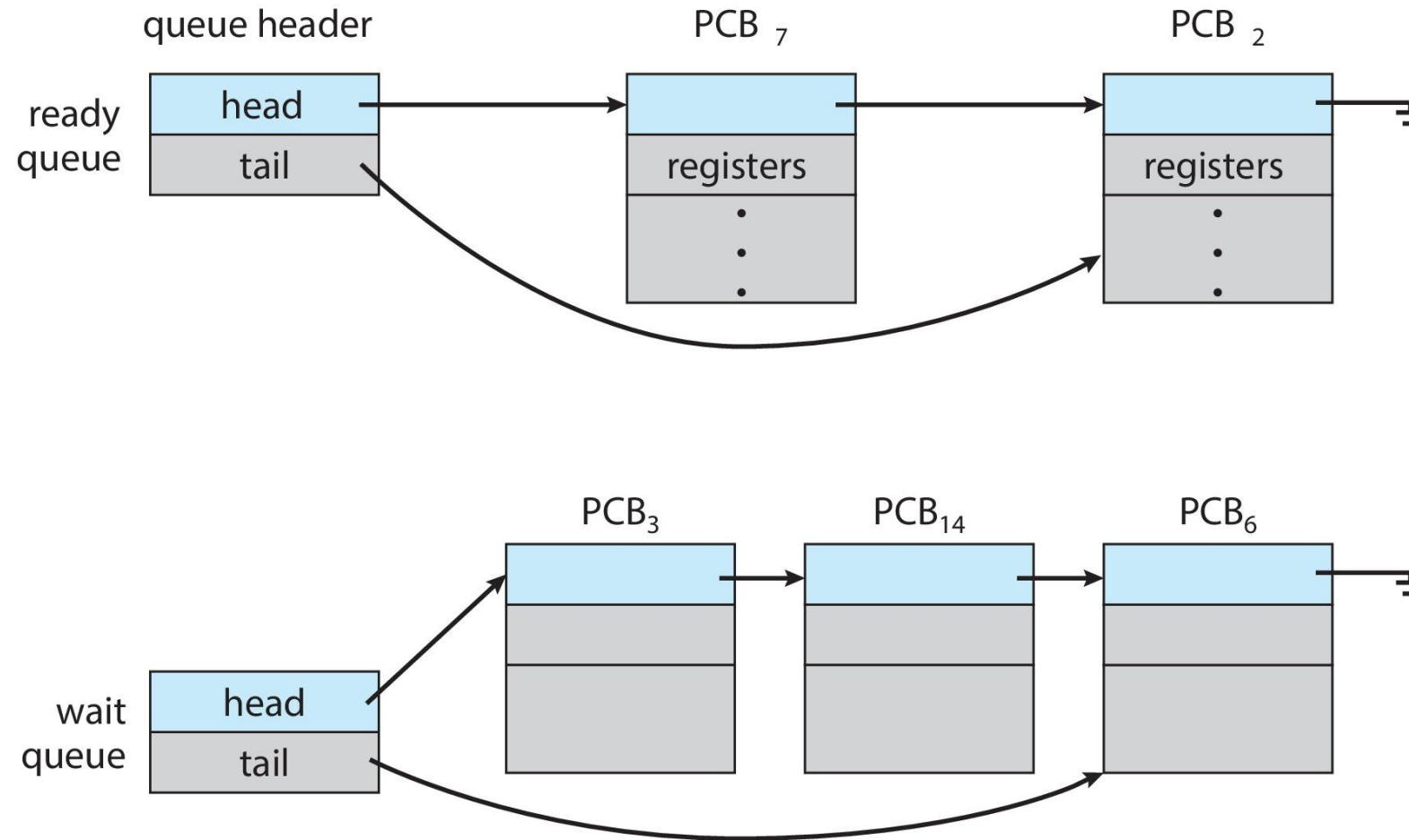


- The four transitions can be indicated as follows
 - Block running → block
 - Timerunout running → ready
 - Dispatch ready → running
 - Wakeup block → ready

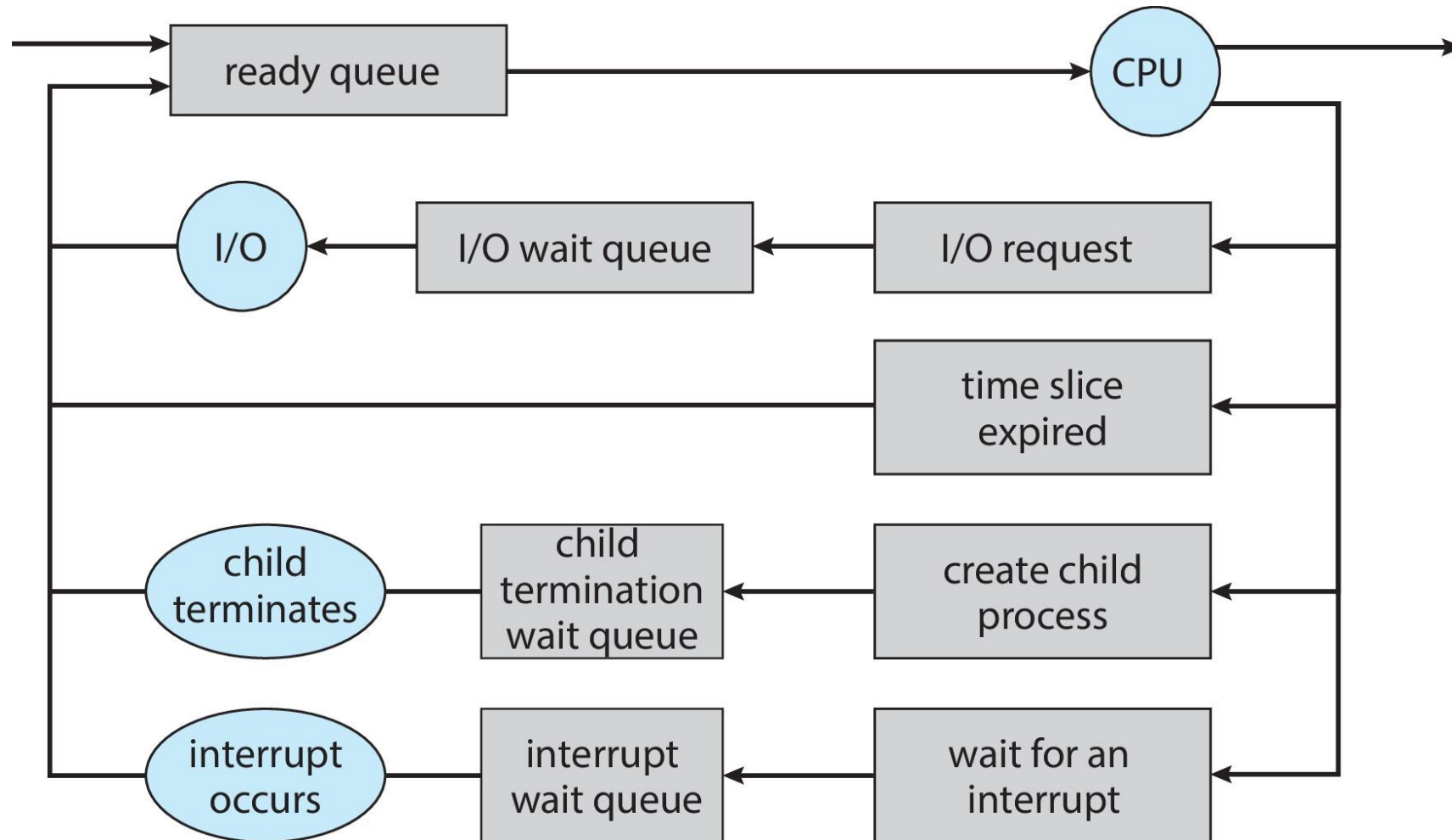
Process Scheduling

- Process scheduler selects among available processes for next execution on CPU core
- **Goal:** Maximize CPU use, quickly switch processes onto CPU core
- Maintains scheduling queues of processes
 - Ready queue
 - set of all processes residing in main memory, ready and waiting to execute
 - Wait queues
 - set of processes waiting for an event (i.e., I/O)
 - Processes migrate among the various queues

Process Scheduling: Ready and Wait Queues



Process Scheduling Representation



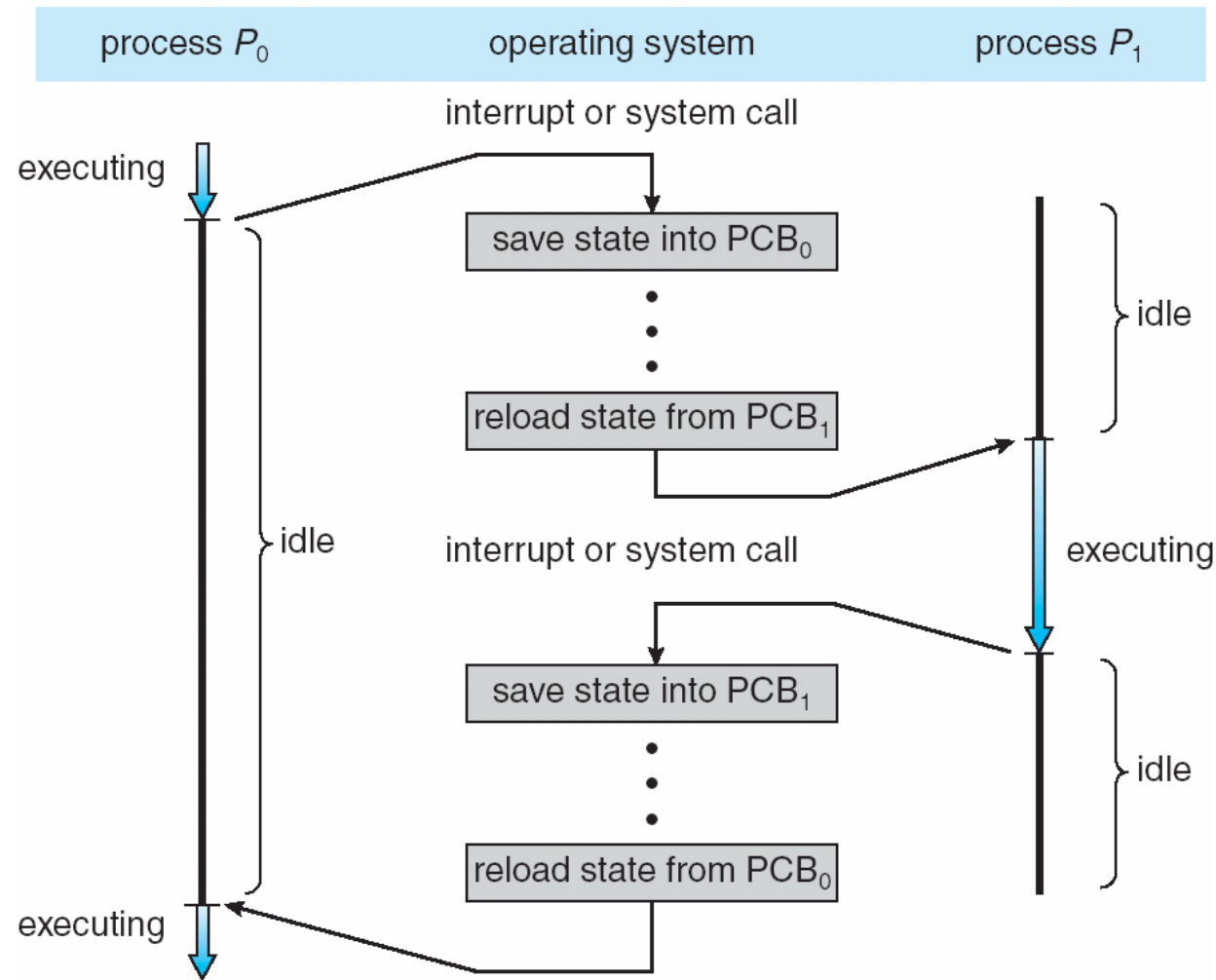
Context Switch

- Context switching is the procedure of switching of process from processor to memory and vice versa in a multiprogramming environment
- When CPU switches to another process
 - The system must save the state of the old process
 - Load the saved state of the new process
- Context switch involves storing and loading the values of each of the process's control block, and process registers, such as;
 - The program counter (Holds the **address of the next instruction** to be executed. After an instruction executes, the PC updates to the next one.)
 - Index register (Holds the **current instruction** being decoded and executed.)
 - General register (Used to **store data temporarily** (numbers, results, or intermediate values) during calculations)
 - Stack pointer etc (Holds an **index value** used for addressing memory locations)

Context Switch

- Context switch is pure overhead
 - The system does no useful work while switching
- Context switch time depends on hardware support e.g, memory speed, the number of registers etc.
- Its range is from 1 to 100 microsecond
 - This speed vary from system to system

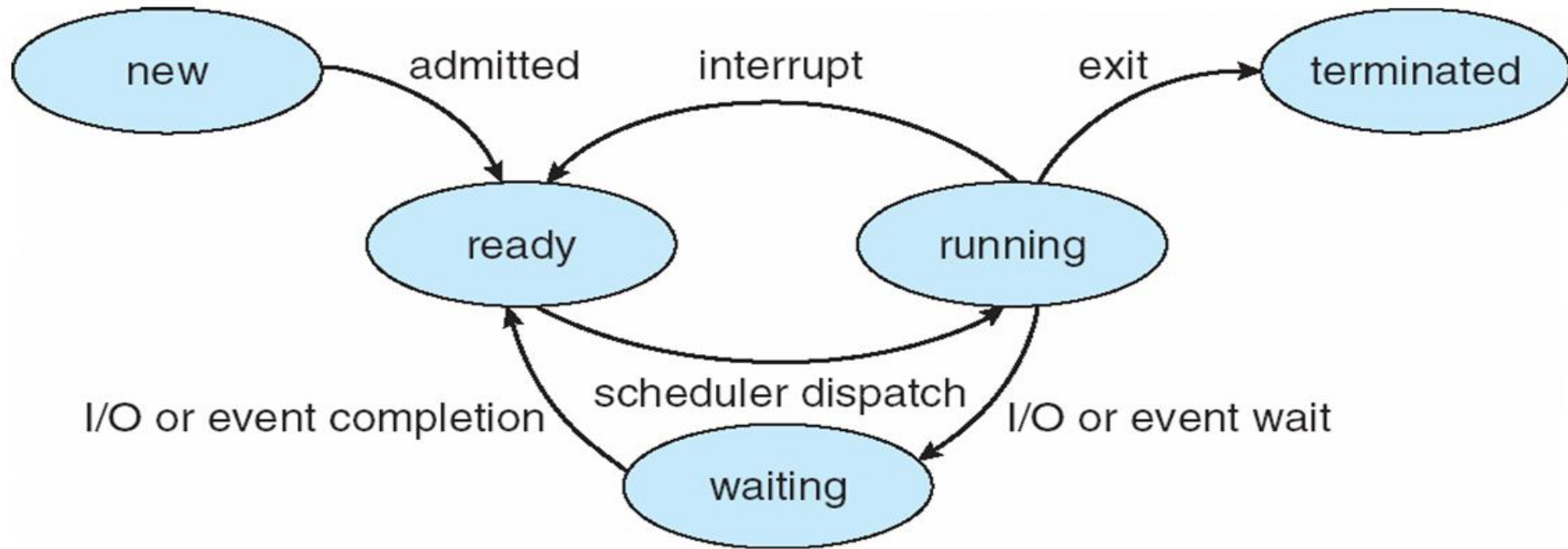
Context Switch



Process Life Cycle

- As a process executes, it changes state
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a process
 - **terminated**: The process has finished execution

Process Life Cycle



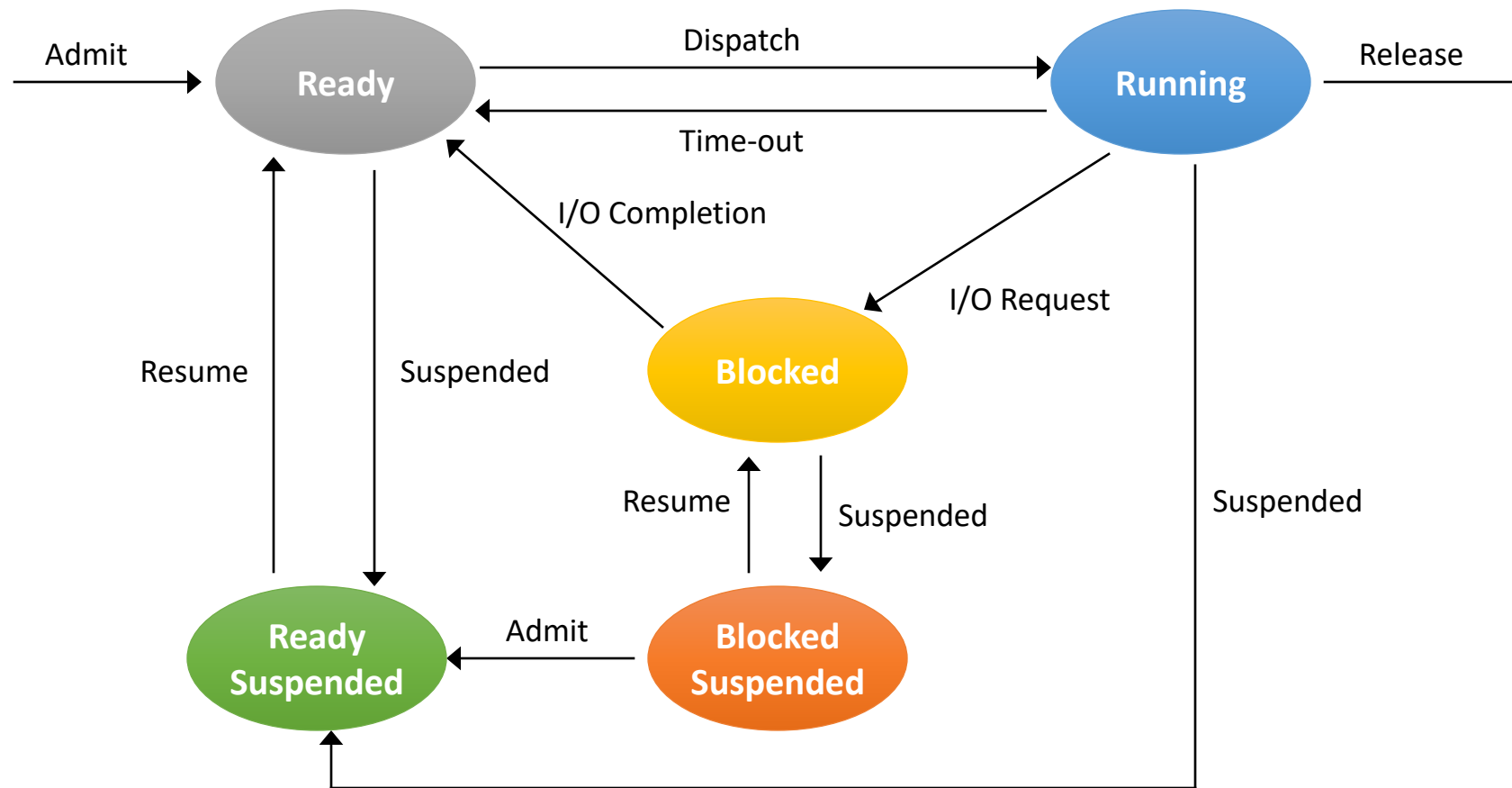
Process Life Cycle

- Assume that process is running with a Unix operating system with no user process
- Various events in life of processes within a multiprogramming system are given below;
 - User using the shell command interpreter, types in a program name, say solvit
 - The shell finds this program and program code will be loaded
 - A system call will be used to generate a process corresponding to the execution of the solvit program
 - The OS creates process control block in the memory
 - The process solvit will now begin to run – the process is said to be in running state
 - After a while, solvit needs to read some data from a disk file and issue an appropriate system call
 - The process is now said to be in the blocked state

Process Life Cycle

- In the meantime, another user wants to run a program called **myprog**
- A new process is created for myprog and since solvit is currently idle, execution begins; myprog is now running
- The I/O delay which is blocking solvit now ends and solvit wants to restart; solvit is now said to be in ready state
- The OS scheduler now decides that myprog has enough processor time (**Timeout**) and moves it into the ready queue
- Solvit is restarted and enters the running state once more
- Switching between active processes, waiting for I/O transfer etc. will continue for the life of the processes
- Eventually, solvit completes its tasks and terminates; it leaves the running state and disappears from the system
- Similarly, myprog terminates

Five State Process Model



Five State Process Model

- There are two queues; a ready queue and a blocked queue
- OS chooses a process the ready queue to run
- In the absence of any priority scheme, this can be a simple FIFO
- A running process is removed from execution, it is either
 - Terminated
 - Time quantum is over
 - Performed I/O
- After I/O completion processes in blocked queue are moved to ready queue
- When an event occurs, the OS must scan the entire blocked queue
- In a large OS, there could be hundreds or even thousands of processes in that queue
- It would be more efficient to have a number of queues, one for each event
- When an event occurs, the entire list of processes in the appropriate queue can be moved to the ready state

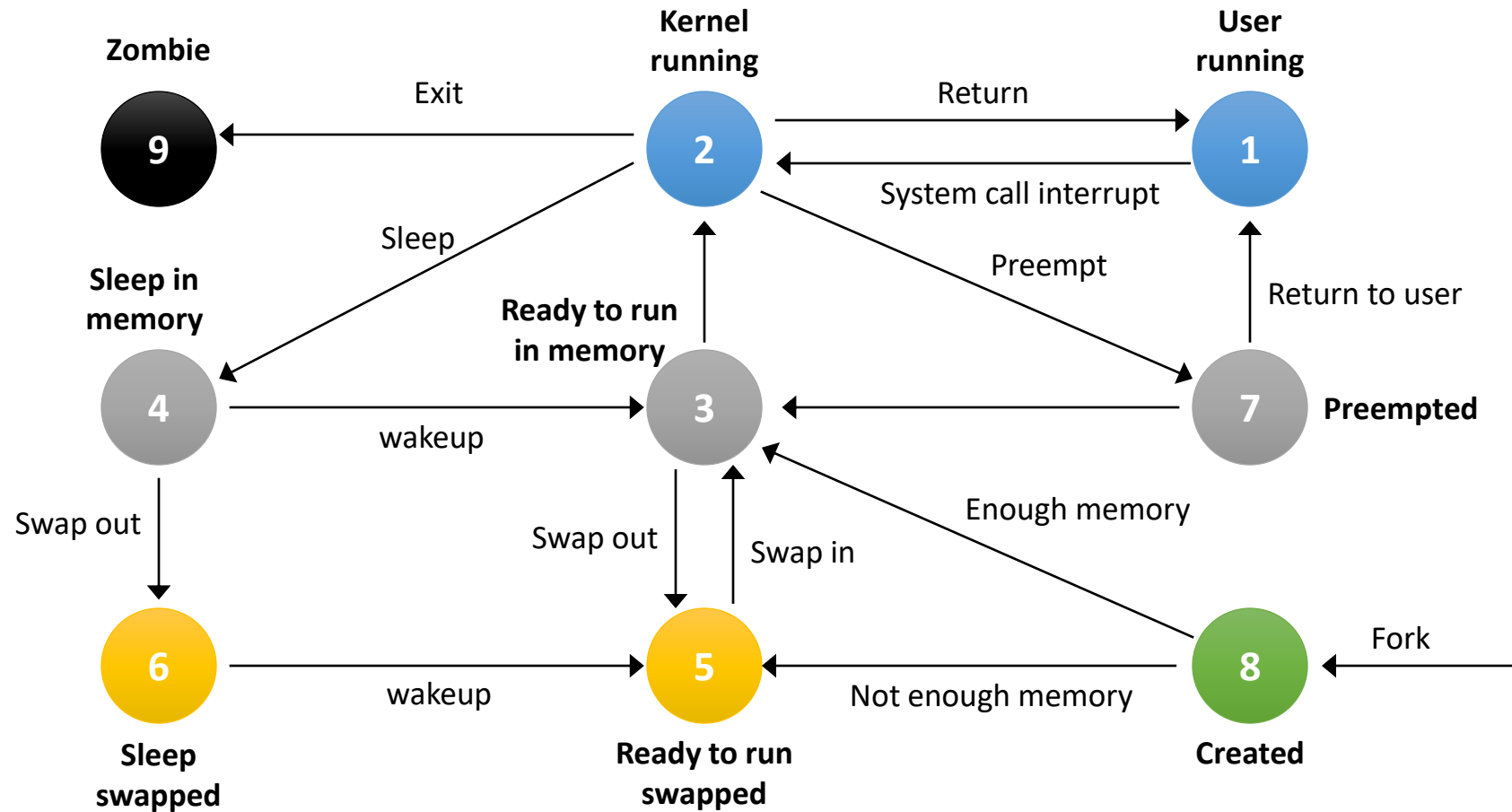
Five State Process Model

- Consider a system that does not employ virtual memory
 - Each process to be executed must be loaded fully into memory
 - In multiple blocked queuing model, all the processes in all the queues must be resident in memory
- Memory holds multiple processes
- Solutions
 - Expand the memory
 - it has own flaw
 - Cost associated with expansion
 - Swapping
 - Moving part or all of a process (blocked) from memory to disk (suspended queue)
- The space that is freed up in memory can be then be used to bring another process
- When a process is suspended, it becomes dormant and merely waits until it is resumed by the system or user

Five State Process Model

- A process can be suspended for the following reasons;
 - Process being swapped out of the memory by the memory management system in order to free memory for higher priority processes
 - A user may wish to suspend execution of a program for purpose of debugging a program
 - Process investigates previous effects of the process
 - Processes designed to run periodically to monitor system usage
 - The OS may suspend a background or utility process or a process that is suspected of causing a problem
 - The process can be suspended while in running, ready or blocked state

Unix Process State Transition



Unix Process State Transition

- Nine process states are recognized by the Unix System V operating system
- Unix employ two running states to indicate weather the process is executing in user mode or kernel mode
- Unix process states are;
 - User running
 - Executing in user mode
 - Kernel running
 - Executing in kernel mode
 - Ready to run (in memory)
 - Ready to run as soon as kernel schedules it
 - Sleeping in memory
 - Unable to run until an event occurs
 - Ready to run (swapped)
 - The swapper must swap the process into memory before the kernel can schedules it to execute

Unix Process State Transition

- Sleeping (swapped)
 - The process is waiting for an event, swapped to a disk
- Preempted
 - Process is returning from kernel to user mode, but the kernel preempts it and does a context switch to schedule another process
- Created
 - Process is newly created and not yet ready to run
- Zombie
 - Process no longer exist, but it leaves a record for its parent process
- A distinction is made between two states; ready to run, in memory and preempted
 - These are essentially the same states as indicated by the dotted line
 - Preemption can occur when a process is about to move from kernel mode to user mode